# FRED TALKS

8th July 2026

## CONTENTS

A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption
UMANG · 1871 WORDS

Zen of AI Coding
YOAV AVIRAM · 1912 WORDS

# A PM's Guide to AI Agent Architecture: Why Capability Doesn't Equal Adoption

UMANG · 05 SEP 2025 · SOURCE

Last week, I was talking to a PM who'd in the recent months shipped their AI agent. The metrics looked great: 89% accuracy, sub-second respond times, positive user feedback in surveys. But users were abandoning the agent after their first real problem, like a user with both a billing dispute and a locked account.

> "Our agent could handle routine requests perfectly, but when faced with complex issues, users would try once, get frustrated, and immediately ask for a human."

This pattern is observed across every product team that focuses on making their agents "smarter" when the real challenge is making architectural decisions that shape how users experience and begin to trust the agent.

In this post, I'm going to walk you through the different layers of AI agent architecture. How your product decisions determine whether users trust your agent or abandon it. By the end of this, you'll understand why some agents feel "magical" while others feel "frustrating" and more importantly, how PMs should architect for the magical experience.

We'll use a concrete customer support agent example throughout, so you can see exactly how each architectural choice plays out in practice. We'll also see why the counterintuitive approach to trust (hint: it's not about being right more often) actually works better for user adoption.

## Let's say you're building a customer support agent

You're the PM building an agent that helps users with account issues - password resets, billing questions, plan changes. Seems straightforward, right?

But when a user says ***"I can't access my account and my subscription seems wrong"*** what should happen?

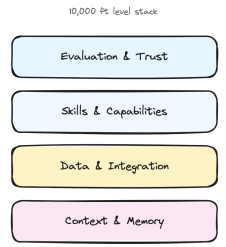
**Scenario A:** Your agent immediately starts checking systems. It looks up the account, identifies that the password was reset yesterday but the email never arrived, discovers a billing issue that downgraded the plan, explains exactly what happened, and offers to fix both issues with one click.

**Scenario B:** Your agent asks clarifying questions. "When did you last successfully log in? What error message do you see? Can you tell me more about the subscription issue?" After gathering info, it says "Let me escalate you to a human who can check your account and billing."

**Same user request. Same underlying systems. Completely different products.**

## The Four Layers Where Your Product Decisions Live

Think of agent architecture like a stack where each layer represents a product decision you have to make.




### Layer 1: Context & Memory (What does your agent remember?)

**The Decision:** How much should your agent remember, and for how long?

*This isn't just technical storage - it's about creating the illusion of understanding. Your agent's memory determines whether it feels like talking to a robot or a knowledgeable colleague.*

**For our support agent:** Do you store just the current conversation, or the customer's entire support history? Their product usage patterns? Previous complaints?

**Types of memory to consider:**

- **Session memory:** Current conversation ("You mentioned billing issues earlier...")
- **Customer memory:** Past interactions across sessions ("Last month you had a similar issue with...")
- **Behavioral memory:** Usage patterns ("I notice you typically use our mobile app...")
- **Contextual memory:** Current account state, active subscriptions, recent activity

The more your agent remembers, the more it can anticipate needs rather than just react to questions. Each layer of memory makes responses more intelligent but increases complexity and cost.

**The Decision:** Which systems should your agent connect to, and what level of access should it have?

*The deeper your agent connects to user workflows and existing systems, the harder it becomes for users to switch. This layer determines whether you're a tool or a platform.*

**For our support agent:** Should it integrate with just your Stripe's billing system, or also your Salesforce CRM, ZenDesk ticketing system , user database, and audit logs? Each integration makes the agent more useful but also creates more potential failure points - *think API rate limits, authentication challenges, and system downtime.*

## Layer 3: Skills & Capabilities (What makes you different?)

**The Decision:** Which specific capabilities should your agent have, and how deep should they go?

*Your skills layer is where you win or lose against competitors. It's not about having the most features - it's about having the right capabilities that create user dependency.*

**For our support agent:** Should it only read account information, or should it also modify billing, reset passwords, and change plan settings? Each additional skill increases user value but also increases complexity and risk.

**Agent-first product strategy**
I help companies shift products from human-centric interfaces to agent-accessible, and where it makes sense, agents-first. That means clarifying what agents should be able to do, what constraints they must operate under, and what needs to change in your APIs, permissions, reliability model, and product surface area to support them.

This typically results in an agent-access roadmap, an operating model, and a prioritized set of product changes that make agents a first-class user.

I'm the co-founder of WhiteBoar, where AI agents build your brand and launch visually striking marketing websites, complete with professional multi-lingual content, in minutes, not months. Contact me at yoav@whiteboar.it.

We are building an agent-first CMS. If we succeed, humans will not need to use it, though they can.

AX is the new UX

Do not assume competence implies perfection.

Like an engineer overseeing the construction of a bridge, your job is not to lay bricks. It is to ensure the structure does not collapse.

Agents will make mistakes. They will drift off course. They will introduce subtle flaws. They will create security vulnerabilities. One of mine attempted to commit a .env file to git. They can be inventive in how they self-sabotage.

Be one step ahead. Ask yourself how this could break. Where could it leak. What could it expose. What assumptions is the agent quietly making.

Play the what-if game relentlessly. Then build guardrails.

Automate checks. Lock down permissions. Isolate environments. Add monitoring. Create recovery paths. Make rollback trivial.

Remember, code is cheap, and you have new superpowers. Build tools to allow you to probe deep, discover vulnerabilities, and test for anticipated failure modes. Design systems that expect failure and degrade gracefully.

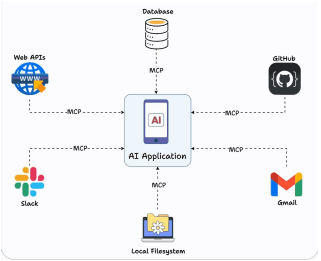
This is the distinction between software development and software engineering.

I work with engineering and product teams adopting agents. The goal is simple: ship faster without lowering standards.

I usually help in two ways:

**Agent-first software engineering**
We turn agents into part of the system, not a novelty. We design workflows, guardrails, and supporting tools so the team can delegate confidently.

If you want to operationalize coding agents without turning your codebase into a casino, get in touch.




**Implementation note:** Tools like MCP (Model Context Protocol) are making it much easier to build and share skills across different agents, rather than rebuilding capabilities from scratch.

## Layer 4: Evaluation & Trust (How do users know what to expect?)

**The Decision:** How do you measure success and communicate agent limitations to users?

*This layer determines whether users develop confidence in your agent or abandon it after the first mistake. It's not just about being accurate - it's about being trustworthy.*

**For our support agent:** Do you show confidence scores ("I'm 85% confident this will fix your issue")? Do you explain your reasoning ("I checked three systems and found...")? Do you always confirm before taking actions ("Should I reset your password now?")? Each choice affects how users perceive reliability.

**Trust strategies to consider:**

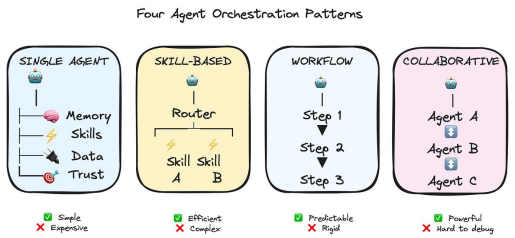
- **Confidence indicators:** "I'm confident about your account status, but let me double-check the billing details"
- **Reasoning transparency:** "I found two failed login attempts and an expired payment method"
- **Graceful boundaries:** "This looks like a complex billing issue - let me connect you with our billing specialist who has access to more tools"
- **Confirmation patterns:** When to ask permission vs. when to act and explain

> The counterintuitive insight: users trust agents more when they admit uncertainty than when they confidently make mistakes.

## So how do you actually architect an agent?

Okay, so you understand the layers. Now comes the practical question that every PM asks: "How do I actually implement this? How does the agent talk to the skills? How do skills access data? How does evaluation happen while users are waiting?"

*Your orchestration choice determines everything about your development experience, your debugging process, and your ability to iterate quickly.*




Lets walk through the main approaches, and I'll be honest about when each one works and when it becomes a nightmare.

### 1. Single-Agent Architecture (Start Here)

Everything happens in one agent's context.

**For our support agent:** *When the user says "I can't access my account," one agent handles it all - checking account status, identifying billing issues, explaining what happened, offering solutions.*

**Why this works:** Simple to build, easy to debug, predictable costs. You know exactly what your agent can and can't do.

**Why it doesn't:** Can get expensive with complex requests since you're loading full context every time. Hard to optimize specific parts.

> Most teams start here, and honestly, many never need to move beyond it. If you're debating between this and something more complex, start here.

You have a router that figures out what the user needs, then hands off to specialized skills.

Refactoring is cheap, but it is cheaper to detect flawed direction early.

Discipline matters more, not less, when execution is frictionless.

#### Software is a liability, a product is an asset

Software costs money to maintain. It only becomes an asset when it produces value for someone.

This was always true. It is harder to remember now because building is so easy.

It is tricker then ever to resist the temptation to add features. Resist it. Build what is used. Kill what is not.

Feature gaps close quickly. What once took years to replicate can now take months. What took months can now take days.

A feature set is no longer a durable moat.

Moats shift toward distribution, data, brand, network effects, regulation, and ecosystem integration.

The barrier to entry falls. The barrier to defensibility rises.

There is a new hierarchy.

At the base are models (Opus 4.6, GPT-5.3-codex). On top of them sit agents (Claude Code, OpenClaw). On top of agents sit services.

Today, agents use services designed for humans. It works, but it is inefficient. The next step is services designed for agents (Moltbook being the harbinger).

Make your services accessible to agents. Expose structured context. Provide machine-readable surfaces.

Agentic commerce is emerging. In some cases, a markdown endpoint is enough. In others, the service itself must be designed agent-first.

Agent-first is not a tooling choice. It is a product decision. It changes your interfaces, your reliability model, your metrics, and what you build next.

Yet when running multiple agents in parallel, the real bottleneck is human coordination.

You must track what each agent is doing. You must remember which assumptions were made. You know what questions to ask next.

As agents become better at managing their own context, your ability to supervise them becomes the limiting factor.

The constraint is no longer tokens. It is cognition.

### Build for a changing world

New models. New capabilities. New frameworks. New skills.

The ground shifts daily. Some models are better at reasoning. Some at coding. Some at translation.

Flexibility is not optional. It must be built into your workflow and products. Both must be engineered for experimentation and adaptability.

### When considering Buy vs. Build, the answer, more often, is build

When code was expensive, buying made sense. When code approaches zero marginal cost, the calculus changes.

Every paid API is now a question. Could we replicate this internally? Should we?

For our QA agent, we needed full-page screenshots of live sites. Many APIs offered this. Instead, we deployed Playwright to AWS Lambda, tailored to our needs, and hardened.

This is not a blanket argument against SaaS. Maintenance, security, and scale still matter. But many small and medium services that once required vendors are now within reach.

Some call this the end of SaaS.

### Fast rubbish is still rubbish

Speed is intoxicating. It is also dangerous.

You can generate enormous amounts of code quickly. But velocity and lines of code written were always terrible indicators of progress.

**For our support agent:** *Router realizes this is an account access issue and routes to the `LoginSkill`. If the LoginSkill discovers it's actually a billing problem, it hands off to `BillingSkill`.*

**Real example flow:**

1. *User: "I can't log in"*
2. *Router → LoginSkill*
3. *LoginSkill checks: Account exists ✓, Password correct ✗, Billing status... wait, subscription expired*
4. *LoginSkill → BillingSkill: "Handle expired subscription for user123"*
5. *BillingSkill handles renewal process*

**Why this works:** More efficient - you can use cheaper models for simple skills, expensive models for complex reasoning. Each skill can be optimized independently.

**Why it doesn't:** Coordination between skills gets tricky fast. Who decides when to hand off? How do skills share context?

> Here's where MCP really helps - it standardizes how skills expose their capabilities, so your router knows what each skill can do without manually maintaining that mapping.

You predefine step-by-step processes for common scenarios. Think LangGraph, CrewAI, AutoGen, N8N, etc.

**For our support agent:** *"Account access problem" triggers a workflow:*

1. *Check account status*
2. *If locked, check failed login attempts*
3. *If too many failures, check billing status*
4. *If billing issue, route to payment recovery*
5. *If not billing, route to password reset*

**Why this works:** Everything is predictable and auditable. Perfect for compliance-heavy industries. Easy to optimize each step.

**Why it doesn't:** When users have weird edge cases that don't fit your predefined workflows, you're stuck. Feels rigid to users.

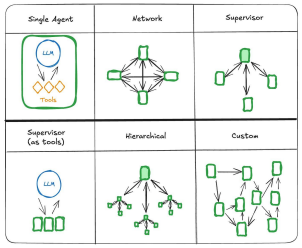
Multiple specialized agents work together using A2A (agent-to-agent) protocols.

**The vision:** Your agent discovers that another company's agent can help with issues, automatically establishes a secure connection, and collaborates to solve the customer's problem. *Think a booking.com agent interacting with an American Airlines agent!*

**For our support agent:** *`AuthenticationAgent` handles login issues, `BillingAgent` handles payment problems, `CommunicationAgent` manages user interaction. They coordinate through standardized protocols to solve complex problems.*

**Reality check:** This sounds amazing but introduces complexity around security, billing, trust, and reliability that most companies aren't ready for. We're still figuring out the standards.

This can produce amazing results for sophisticated scenarios, but debugging multi-agent conversations is genuinely hard. When something goes wrong, figuring out which agent made the mistake and why is like detective work.




But here's what's interesting - even with the perfect architecture, your agent can still fail if users don't trust it. That brings us to the most counterintuitive lesson about building agents.

## The trust thing that everyone gets wrong

Here's something counterintuitive: Users don't trust agents that are right all the time. They trust agents that are honest about when they might be wrong.

Good tests are the first feedback loop. The agent will iterate until the tests pass. It is your responsibility to make sure the tests are adequate (by instructing the agent to create good ones) and to make sure the agent does not cheat (by weakening the tests or simply skipping them).

Give your agent access to your CI to create a second feedback loop. Give it access to your server logs to create a third. Giving the agent away to visually inspect a UI is yet another example.

Your job is to design environments where iteration converges toward correctness instead of drifting toward plausible nonsense.

Velocity without feedback is chaos.

### Any stack is your stack

Agents are competent across most major stacks where training data exists. Since you are not writing the code, your attachment to a specific stack becomes less relevant.

Do not limit yourself to what you personally know. Your conceptual understanding transfers more than you think.

When you lack terminology or domain knowledge, ask the agent to brief you. The barrier to entry into unfamiliar ecosystems is lower than it has ever been.

### Agents are not just for coding

Coding is only the beginning.

Agents can assist with business analysis, UX, infrastructure, operations, ad campaigns, analytics configuration, even accounting workflows.

I migrated four WordPress blogs from an overpriced host to Hetzner in minutes after postponing the task for years. The blocker was never a technical difficulty. It was attention. Claude completed the task in 15 minutes.

Agents reduce the activation energy required to execute tedious but valuable work.

Grant access carefully. Limit scope. Audit results. Direction without oversight is reckless.

### The context bottleneck is in your head

Context engineering has improved. Tooling has improved.

Making imperfect decisions is no longer fatal. In fact, it can be productive. A flawed reference implementation provides better context than a pristine specification. Agents reason more effectively from concrete artifacts than from abstract intent.

Rapid iteration is now the default mode. Over the last three months, we rebuilt our CMS four times from scratch, each with a different architecture. Each time we learned more about what we actually needed and what works.

When code is cheap, you can run more small bets.

### So is repaying technical debt

There is little excuse for stale dependencies or ignored security patches. Updating libraries, migrating APIs, modernizing patterns. These are prompts away.

Prune your code regularly. Upgrade aggressively. Keep the surface clean.

Technical debt has not disappeared. But the cost of servicing it has dropped. That changes the incentive structure. Neglect becomes less defensible.

### All bugs are shallow

Linus's law states that "given enough eyeballs, all bugs are shallow." Those eyeballs are now abundant.

Ask one model to review your code. Ask another. They may find different issues. Ask them to fix what they find. Often they will succeed.

How "dense" are bugs given a piece of code is a philosophical question. Bugs are not magically gone. Agents do not achieve perfection. From a practical perspective, however, the limiting factor is no longer the number of reviewers. It is the tightness of your feedback loops.

The claim here is profound: comprehension of the codebase at the function level is no longer necessary. At the same time, relying entirely on the models is a recipe for disaster. You still have a job! (see: create tight feedback loops, anticipate modes of failure)

Agents can sometimes one-shot a task.

We've built agents that create a complete custom-made marketing website in one shot, but this is by far the exception. In most cases, agents operate best when iterating towards a well-defined goal, guided by feedback.

Think about it from the user's perspective. Your support agent confidently says "I've reset your password and updated your billing address." User thinks "great!" Then they try to log in and... it doesn't work. Now they don't just have a technical problem - they have a trust problem.

Compare that to an agent that says "I think I found the issue with your account. I'm 80% confident this will fix it. I'm going to reset your password and update your billing address. If this doesn't work, I'll immediately escalate to a human who can dive deeper."

**Same technical capability. Completely different user experience.**

Building trusted agents requires focus on three things:

1. **Confidence calibration:** When your agent says it's 60% confident, it should be right about 60% of the time. Not 90%, not 30%. Actual 60%.

2. **Reasoning transparency:** Users want to see the agent's work. "I checked your account status (active), billing history (payment failed yesterday), and login attempts (locked after 3 failed attempts). The issue seems to be..."

3. **Graceful escalation:** When your agent hits its limits, how does it hand off? A smooth transition to a human with full context is much better than "I can't help with that."

A lot of times we obsess over making agents more accurate, when what users actually want was more transparency about the agent's limitations.

## What's Coming Next

In Part 2, I'll dive deeper into the autonomy decisions that keep most PMs up at night. How much independence should you give your agent? When should it ask for permission vs forgiveness? How do you balance automation with user control?

We'll also walk through the governance concerns that actually matter in practice - not just theoretical security issues, but the real implementation challenges that can make or break your launch timeline.

# Zen of AI Coding

YOAV AVIRAM · 08 MAR 2026 · SOURCE

Dedicated to all those who are sceptical about the significance of agentic coding, and to those who are not, and are wondering what it means for the future of their profession. The title is an homage to Zen of Python by Tim Peters. Unlike Tim, I am not a zen master. My only aim is to take stock of where we are and where we might be heading. I have been building with coding agents daily for the past year, and I also help teams adopt them without losing reliability or security.

1. Software development is dead
2. Code is cheap
3. Refactoring easy
4. So is repaying technical debt
5. All bugs are shallow
6. Create tight feedback loops
7. Any stack is *your* stack
8. Agents are not just for coding
9. The context bottleneck is in your head
10. Build for a changing world
11. When considering Buy vs. Build, the answer, more often, is build
12. Fast rubbish is still rubbish
13. Software is a liability, a product is an asset
14. Moats are more expensive
15. Build for agents
16. Anticipate modes of failure

### Software development is dead

You do not need to write another line of code if you do not want to. Coding agents can accomplish most coding tasks with the right direction.

Software development, as the act of manually producing code, is dying. A new discipline is being born. Your role in it is vital, but it is no longer centered on typing code. It is centered on framing problems, shaping context, defining constraints, and judging outcomes.

The marginal cost of code is collapsing. That single fact changes everything that follows.

### Code is cheap

The economics of software have changed.

When coding is cheap, implementation stops being the constraint. You can build ten things in parallel. You cannot decide, validate, and ship ten things in parallel, at least not without changing the rest of the pipeline.

Cost of delay shifts. It is no longer about developer days. It is about time stuck in other bottlenecks: product decisions, unclear requirements, security review, user testing, release processes, and operational risk. Agents can flood these queues. Inventory grows. Lead time grows. Delay becomes more expensive, not less.

This changes prioritization. The highest leverage work is what unblocks shipping: tight feedback loops, tests, evals, guardrails, observability, and clear acceptance criteria. Anything that turns “we can build it” into “we can trust it”.

Agents can help alleviate some of these bottlenecks. The challenge is applying them outside of coding (see Agents are not just for coding)

### Refactoring is easy

The cost of changing your mind is lower than it has ever been. Architectural decisions that once felt permanent are now provisional.

You chose React. Two months later, you regret it. Ask an agent to rewrite the project.